
CLUSTER

Backend Engineer Task Sheet

Spring Boot API — V4 Design Alignment

CONFIDENTIAL — Engineering Only

June 2026

Executive Summary

The v4 design package introduces 29 backend-dependent features. After auditing the Spring Boot API (Clusteer-API), 12 are fully implemented, 10 are partially there, and 17 are completely missing. This document specifies exactly what the backend engineer needs to build, separated into **SAFE features** (no money movement) and **FINANCIAL features** (touches money — requires extra caution and review).

Current Backend Readiness

Status	Count	Examples
EXISTS	12	Buy/sell orders, OTP, 2FA, notifications, password change
PARTIAL	10	Exchange rate (stub), sessions (stored, no endpoint), fee config (unused)
MISSING	17	Support tickets, referrals, payment requests, change email, API keys, CMS, compliance

Section 1: Critical Fixes (Still Open from Previous Audit)

These were flagged in the May 2026 audit. Confirm they are resolved before proceeding.

1.1. Webhook event filter is INVERTED

File: QuidaxServiceImpl.java:267

The if-condition rejects valid events and accepts invalid ones. Negate the condition: `if (!List.of(...).contains(event))`. Without this fix, no transaction completes.

1.2. Webhook endpoint blocked by JWT auth

File: AppConfig.java:56-62

Add `/v1/webhook/**` to the `permitAll()` list. Quidax callbacks use HMAC, not JWT.

1.3. Exchange rate endpoint returns empty

File: PreferenceServiceImpl.java:218-225

`getExchangeRate()` creates an `ExchangeRateResponse` but never populates it. Implement using Quidax rate API or CoinGecko + forex.

1.4. Fee config not applied in trade flow

File: P2POrderServiceImpl.java

Preference entity has `purchaseMarkUp/saleMarkDown` fields but the trade flow ignores them — fees come only from Quidax `blockchain_fee`. Decide: is Quidax the fee authority, or should internal markup be applied on top? Document the decision.

Section 2: Safe Features (No Money Movement)

These features involve CRUD operations on non-financial entities. They do not touch order processing, Quidax API calls, transaction balances, or webhook handling. Safe to implement independently.

2.1 Support Ticket System

Priority: P2 | Effort: 2 days

New entities, repository, service, and controller needed.

Component	Details
Entity: SupportTicket	id, userId, ticketNumber (unique, e.g. CLR-8473), subject, category (enum: BUY_SELL, KYC, PAYMENTS, AC
Entity: TicketMessage	id, ticketId (FK), senderId, senderRole (CUSTOMER/AGENT/SYSTEM), body (5000c), attachmentUrl, createdA
Controller: /v1/support	POST / (create ticket), GET / (list user tickets), GET /{ticketNumber} (detail + messages), POST /{ticketNumber}
Admin endpoints	GET /v1/support/all (paginated, filterable by status/priority), PUT /v1/support/{ticketNumber}/assign, PUT /v1/sup
Auth	CUSTOMER for own tickets, ADMIN/SUPER for all tickets

2.2 Change Email Endpoint

Priority: P1 | Effort: 4 hours

Component	Details
Endpoint	PUT /v1/user/email/update
Request body	{ newEmail: string, password: string }
Flow	1. Validate password against stored hash. 2. Check newEmail uniqueness. 3. Generate verification token. 4. Send verification
Entity change	Add EmailChangeRequest entity or reuse EmailVerification with type=EMAIL_CHANGE
Auth	CUSTOMER

2.3 Active Sessions Endpoint

Priority: P1 | Effort: 2 hours

AccountSecurity already stores savedDevices and savedOrigins as JSON strings. Expose this data and add a revoke endpoint.

Component	Details
GET /v1/user/sessions	Parse savedDevices JSON, return list of { deviceName, os, location, ip, lastActive, isCurrent }
DELETE /v1/user/sessions/{deviceId}	Remove device from savedDevices list, invalidate its token
DELETE /v1/user/sessions/all	Clear all sessions except current, invalidate all other tokens
Auth	CUSTOMER

2.4 Payment Request System

Priority: P2 | Effort: 1 day

Component	Details
Entity: PaymentRequest	id, userId, amount (BigDecimal), currency (USDT/USDC), note (500c), linkCode (unique, 8-char alphanumeric), status
Controller: /v1/payment-request	POST / (create, returns linkCode), GET /{linkCode} (public — no auth, returns amount + recipient username), GET / (list)
Link format	clusteer.app/pay/{username}/{linkCode}
Auth	CUSTOMER for create/list, PUBLIC for view by linkCode

2.5 Referral System

Priority: P3 | Effort: 2 days

Component	Details
Entity: Referral	id, referrerId (FK User), referredId (FK User), referralCode, status (INVITED/SIGNED_UP/KYC_VERIFIED/FIRST_TRADE)
User entity change	Add referralCode (unique, 8-char) generated on registration
Controller: /v1/referrals	GET /stats (total invited, signed up, qualified, earned, pending), GET /list (paginated referral list), POST /apply (apply referral)
Reward trigger	When referred user completes first trade, create reward record. Actual payout is manual/admin action.
Auth	CUSTOMER

2.6 API Key Management

Priority: P3 | Effort: 1 day

Component	Details
Entity: UserApiKey	id, userId (FK), name, keyPrefix (first 8 chars, for display), keyHash (SHA-256 of full key), permissions (JSON: {read: boolean})
Controller: /v1/user/api-keys	POST / (create — return full key ONCE), GET / (list — show prefix only), DELETE /{id} (revoke)
Key format	clk_live_{32-char-random} — show once on creation, store only hash
Auth	CUSTOMER

2.7 KYC Tier Model + Source of Funds

Priority: P2 | Effort: 2 days

Component	Details
UserKyc entity changes	Add: tier (int 0-3), sourceOfFunds (enum: SALARY/BUSINESS/INVESTMENTS/SAVINGS/GIFT/OTHER), employmentStatus
Tier rules	Tier 1: BVN/NIN only. Tier 2: + government ID + selfie. Tier 3: + proof of address + source of funds.
Transaction limits	Add to Preference or new AccountLimits entity: tier1DailyLimit, tier1MonthlyLimit, tier2DailyLimit, tier2MonthlyLimit, tier3DailyLimit, tier3MonthlyLimit
Endpoint	PUT /v1/user/kyc/tier-upgrade (submit Tier 3 docs), GET /v1/user/kyc/limits (current tier + usage vs caps)
Note	Limit ENFORCEMENT in the order flow is a FINANCIAL change — build the model now, enforce later with review

2.8 Wire Report Export to Controller

Priority: P1 | Effort: 1 hour

ExportService already has generateAdminExcel() and generateAuditExcel() methods using Apache POI. Wire them to a controller endpoint: GET /v1/system/reports/export?type=transactions|audit&format=xlsx&from=&to=. Return as downloadable file with Content-Disposition header. Auth: SUPER/ADMIN.

2.9 Audit Log Hash Chain

Priority: P3 | Effort: 4 hours

Add previousHash (VARCHAR 64) and recordHash (VARCHAR 64) fields to EventRecord. On each new event: compute SHA-256(previousHash + userId + category + description + timestamp), store as recordHash. Set previousHash to the last event's recordHash. This creates a tamper-evident chain — if any row is modified, all subsequent hashes break. No other changes needed.

2.10 CMS Content Management

Priority: P3 | Effort: 2 days

Component	Details
Entity: CmsContent	id, type (BANNER/ANNOUNCEMENT/FAQ), title, body (5000c), audience (ALL/TIER_1/TIER_2_PLUS/UNVERIFIED/...
Controller: /v1/cms	CRUD: POST, GET /all (admin), GET /active (public — returns published + non-expired), PUT /{id}, DELETE /{id}, PUT
Auth	SUPER/ADMIN for CRUD, PUBLIC for active content

2.11 Staff Permissions

Priority: P3 | Effort: 1 day

Add role enum values: COMPLIANCE, SUPPORT, FINANCE, READ_ONLY (currently only SUPER/ADMIN). Create AdminPermission entity mapping role to permission strings (users.read, kyc.approve, etc.). Add @PreAuthorize checks on sensitive admin endpoints using the permission matrix. Add POST /v1/admin/invite endpoint that sends an email invite link.

2.12 Admin System Settings

Priority: P2 | Effort: 4 hours

Add fields to Preference entity: maintenanceMode (boolean), pauseBuyOrders (boolean), pauseSellOrders (boolean), readOnlyMode (boolean), autoApproveBelow (BigDecimal), manualReviewAbove (BigDecimal), dualApprovalAbove (BigDecimal), coolingPeriodMinutes (int). These are admin-configurable via the existing PUT /v1/system/preferences/update endpoint.

Section 3: Financial Features (Backend Engineer Only)

These features touch money movement, pricing, or security enforcement. They require careful implementation, code review, and testing before deployment. Do NOT implement these without explicit sign-off from the product owner.

Feature	Risk	What to Do
Exchange rate implementation	Wrong rate = wrong prices	Implement <code>getExchangeRate()</code> using Quidax <code>/api/v1/merchants/custodial/rate</code> or CoinGecko +
Fee application decision	Revenue impact	Document whether Quidax <code>blockchain_fee</code> is the only fee, or if internal markup (<code>purchaseMark</code>
Transaction limit enforcement	Regulatory compliance	Once KYC tier model is built (2.7), add limit checks in <code>createPurchaseOrder/createSaleOrder</code> .
Webhook idempotency	Duplicate processing	Add idempotency check in <code>processWebhook</code> — if order already completed/failed, skip. Prevent
Compliance/AML cases	Regulatory requirement	Full case management system — entity, workflow (open/investigating/escalated/filed/closed), S

Section 4: Recommended Implementation Sequence

Week	Tasks	Effort
Week 1	Fix critical bugs (1.1-1.4): webhook filter, webhook auth, exchange rate, fee decision	1 day
Week 1	Wire report export (2.8), expose sessions (2.3), change email (2.2)	1 day
Week 2	Support ticket system (2.1)	2 days
Week 2	Payment request system (2.4)	1 day
Week 3	KYC tier model + source of funds (2.7)	2 days
Week 3	Admin system settings (2.12)	4 hours
Week 4	Referral system (2.5)	2 days
Week 4	API key management (2.6)	1 day
Week 5	CMS (2.10), staff permissions (2.11), audit hash chain (2.9)	3 days
Week 5+	Financial features (Section 3) — with code review	Ongoing

Total safe features: ~3 weeks of work for 1 backend engineer.

Financial features (Section 3) run in parallel with product/compliance sign-off.

Section 5: New Entity Summary

Entity	Table	Key Fields	Section
SupportTicket	support_tickets	ticketNumber, userId, subject, category, priority, status	2.1
TicketMessage	ticket_messages	ticketId (FK), senderId, senderRole, body, attachmentUrl	2.1
PaymentRequest	payment_requests	userId, amount, currency, linkCode, note, status, expiresAt	2.4
Referral	referrals	referrerId, referredId, referralCode, status, rewardNgn	2.5
UserApiKey	user_api_keys	userId, name, keyPrefix, keyHash, permissions (JSON), active	2.6
CmsContent	cms_content	type, title, body, audience, status, publishedAt, expiresAt, views	2.10

Entity Modifications

Entity	New Fields	Section
User	referralCode (unique, 8-char)	2.5
UserKyc	tier (int), sourceOfFunds (enum), employmentStatus (enum), expectedVolume (enum)	2.7
EventRecord	previousHash (VARCHAR 64), recordHash (VARCHAR 64)	2.9
Preference	maintenanceMode, pauseBuyOrders, pauseSellOrders, readOnlyMode, autoApproveBelow, manualReviewAbove, dualApproval	2.7, 2.10
Admin	role expanded: COMPLIANCE, SUPPORT, FINANCE, READ_ONLY	2.11

Repository: github.com/OutbuildHQ/Clusteer-API | Branch: master
Audit date: June 2026 | Based on: V4 Design Package + PRD v2.0